

HELIOS

Sistema Distribuito per il Monitoraggio e l'Analisi di Impianti
Fotovoltaici Industriali



Studente: [Simone Battiato]

Matricola: [1000031017]

Corso di Ingegneria dei Sistemi Distribuiti

Anno Accademico 2025/2026

Sommario

Il progetto descrive l'architettura, le scelte progettuali e l'implementazione di **Helios**, un sistema software distribuito per la gestione intelligente di impianti di produzione energetica rinnovabile. Il sistema simula uno scenario reale di monitoraggio per una Comunità Energetica (CER) o un impianto industriale, gestendo volumi di dati significativi (oltre 225 MWh annui).

L'architettura segue il modello Client-Server RESTful, implementando pattern avanzati quali *Remote Facade*, *DTO*, *Session State* e *Request Batch*. Particolare attenzione è stata posta alla sicurezza (tramite protocollo JWT) e all'integrazione di sistemi di Intelligenza Artificiale Generativa (LLM) per il supporto decisionale.

Indice

1	Introduzione e Contesto	2
1.1	Scenario di Riferimento	2
1.2	Obiettivi del Sistema	2
2	Architettura del Sistema	3
2.1	Panoramica Architetturale	3
2.1.1	Lato Server (Backend)	3
2.1.2	Lato Client (Frontend CLI)	3
3	Design Pattern Implementati	4
3.1	Remote Facade	4
3.2	Data Transfer Object (DTO)	4
3.3	Session State Pattern	4
3.4	Request Batch	5
4	Analisi Dati e Funzionalità	6
4.1	Analisi Economica ed Ecologica	6
4.2	Correlazioni Ambientali (Vento e Umidità)	6
4.3	Intelligenza Artificiale Generativa	6
4.4	Forecasting e Ore di Picco	7
5	Sicurezza e Flusso Dati	8
5.1	Fase 1: Negoziazione del Token (Handshake)	8
5.2	Fase 2: Il Viaggio della Richiesta	8
5.3	Fase 3: Il Filtro JWT (Deep Dive)	9
5.3.1	Dettaglio delle Operazioni	9
6	Implementazione Client CLI	11
6.1	Struttura del Menu	11
6.2	Gestione dello Stato	11
7	Conclusioni	12

Capitolo 1

Introduzione e Contesto

1.1 Scenario di Riferimento

Il progetto nasce per rispondere alle esigenze di un'azienda fittizia operante nel settore logistico che possiede un impianto fotovoltaico da circa 200 kWp installato sulla copertura del proprio stabilimento.

L'azienda necessita di trasformare i dati grezzi provenienti dai sensori (SCADA/IoT) in informazioni strategiche per:

- **Monitoraggio Tecnico:** Rilevare cali di efficienza dovuti a sporcizia o guasti.
- **Controllo Economico:** Calcolare il ROI (Return on Investment) e il risparmio in bolletta.
- **Sostenibilità:** Quantificare le emissioni di CO_2 evitate.

1.2 Obiettivi del Sistema

Helios si propone di risolvere le criticità legate alla gestione passiva dell'impianto. Gli obiettivi principali sono:

1. **Centralizzazione:** Aggregare dati eterogenei (produzione elettrica, dati meteo) in un unico database.
2. **Sicurezza:** Garantire l'accesso ai dati sensibili solo a personale autorizzato tramite token crittografici.
3. **Intelligence:** Fornire analisi predittive e diagnostiche utilizzando algoritmi statistici e IA Generativa.

Capitolo 2

Architettura del Sistema

2.1 Panoramica Architetture

Il sistema è basato su un'architettura distribuita a due livelli (Two-Tier), progettata per garantire disaccoppiamento e scalabilità.

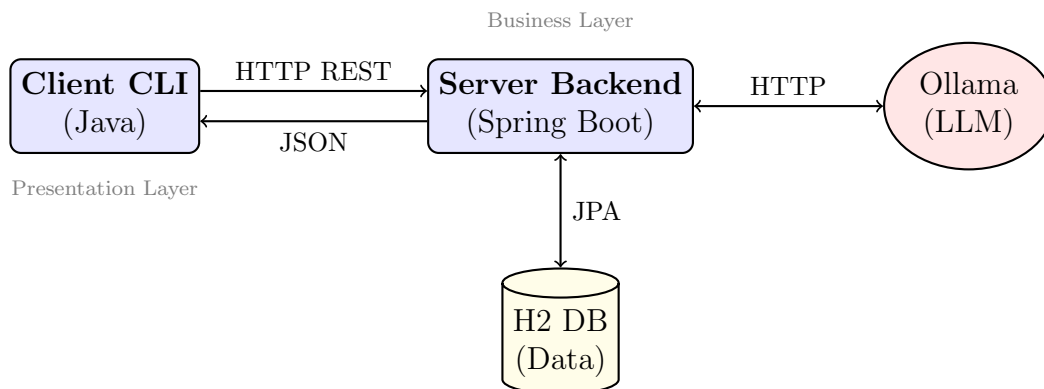


Figura 2.1: Schema ad alto livello dell'interazione tra i componenti del sistema Helios.

2.1.1 Lato Server (Backend)

Il cuore del sistema è sviluppato in **Java** utilizzando il framework **Spring Boot**. Espone una serie di API RESTful che fungono da interfaccia per qualsiasi client autorizzato.

- **Tecnologie:** Java 17, Spring Boot 3, Spring Security, JPA/Hibernate.
- **Database:** H2 Database (In-Memory) per la persistenza veloce dei dati durante la simulazione.
- **Integrazione AI:** Modulo di comunicazione HTTP con **Ollama** (Llama 3 / Mistral) per l'elaborazione del linguaggio naturale.

2.1.2 Lato Client (Frontend CLI)

L'interfaccia utente è una **Command Line Interface (CLI)** avanzata sviluppata in Java puro. Utilizza le librerie 'java.net.http' per comunicare con il backend. La scelta di una CLI enfatizza la natura "Enterprise" e di amministrazione remota del tool, permettendo l'esecuzione anche su server headless.

Capitolo 3

Design Pattern Implementati

Per soddisfare i requisiti di ingegneria del software, sono stati implementati diversi pattern architetturali.

3.1 Remote Facade

Per ridurre la latenza di rete e il numero di chiamate ("chattiness"), il server espone API *Coarse-Grained*. Il controller `EnergyController` agisce da facciata, incapsulando la complessità della logica di business.

- **Esempio:** L'endpoint `/daily-report-session` non restituisce righe grezze del database, ma un report già aggregato e calcolato.

3.2 Data Transfer Object (DTO)

I dati interni del database (Entity `RenewableData`) non vengono mai esposti direttamente. Sono stati creati oggetti specifici per il trasporto dati, come `DailyReportDTO` e `BatchInsightsDTO`. Questo garantisce che modifiche allo schema del database non rompano il contratto con il client.

3.3 Session State Pattern

Sebbene REST sia stateless, è stato implementato un meccanismo di sessione per migliorare l'esperienza utente nelle analisi continuative.

1. Il client invia una richiesta di filtro temporale (es. "Gennaio 2017").
2. Il server memorizza questo contesto e restituisce un cookie di sessione.
3. Il client allega il cookie alle richieste successive, permettendo analisi filtrate senza ripetere i parametri.

3.4 Request Batch

Per ottimizzare la scoperta di informazioni, l'endpoint `/batch-suggestions` esegue molteplici query in parallelo (giorno migliore, giorno peggiore, anomalie) e restituisce un unico pacchetto JSON.

Capitolo 4

Analisi Dati e Funzionalità

Il valore aggiunto di Helios risiede nei suoi algoritmi di analisi. Di seguito le principali metriche implementate.

4.1 Analisi Economica ed Ecologica

Il sistema traduce i kWh prodotti in valore monetario e impatto ambientale.

```
1 double totalValue = totalKwh * 0.20; // Prezzo medio energia
2 double co2Saved = totalKwh * 0.4;    // Fattore emissione mix energetico
```

Listing 4.1: Algoritmo di calcolo ROI e CO2

Dai dati analizzati, l'impianto ha generato un risparmio stimato di oltre **45.000 €** e ha evitato l'emissione di **90 tonnellate di CO₂**.

4.2 Correlazioni Ambientali (Vento e Umidità)

Il sistema analizza come le variabili meteo influenzano la produzione.

- **Impatto Vento:** È stato rilevato che venti superiori a 4 m/s aumentano l'efficienza dei pannelli (raffreddamento naturale), incrementando la produzione media in modo significativo.
- **Impatto Umidità:** Al contrario, alti tassi di umidità sono stati correlati a una diminuzione della resa, spesso associata a foschia o nuvolosità.

4.3 Intelligenza Artificiale Generativa

Tramite l'opzione 9 del Client, il sistema invia un sottoinsieme di dati critici a una LLM locale.

- **Input:** Dati grezzi (es. "Prod: 0, Radiazione: 600").
- **Output:** Report discorsivo (es. "Attenzione: rilevata anomalia critica. Irraggiamento presente ma produzione assente. Probabile guasto inverter.").

4.4 Forecasting e Ore di Picco

Il sistema offre funzionalità predittive basate su medie mobili storiche e identifica le fasce orarie di picco per permettere strategie di *Load Shifting* (spostamento dei carichi industriali).

Capitolo 5

Sicurezza e Flusso Dati

La sicurezza di Helios è progettata attorno al paradigma **Stateless**. A differenza delle sessioni server-side tradizionali (che richiedono memoria sul server per ogni utente attivo), l'architettura RESTful adotta un approccio basato su **Token JWT (JSON Web Token)**.

Questo garantisce la scalabilità orizzontale: ogni richiesta HTTP è autosufficiente e contiene tutte le informazioni necessarie per identificare il richiedente.

5.1 Fase 1: Negoziazione del Token (Handshake)

Il flusso di sicurezza inizia quando un operatore tenta di accedere al sistema. Poiché il server non mantiene memoria delle connessioni passate, è necessaria una fase preliminare di autenticazione.

1. **Invio Credenziali:** Il Client CLI raccoglie username e password e invia una richiesta POST all'endpoint pubblico `/api/auth/login`.
2. **Verifica e Firma:** Il server verifica l'hash della password nel database. Se corretto, genera un token JWT firmato con algoritmo **HMAC-SHA256** contenente:
 - *Subject:* L'identificativo dell'utente (es. "admin").
 - *Expiration:* La data di scadenza (fissata a 24 ore).
 - *Signature:* Una firma crittografica per prevenire manomissioni.
3. **Consegna:** Il token viene restituito al client, che lo memorizza in una variabile locale per le richieste future.

5.2 Fase 2: Il Viaggio della Richiesta

Una volta autenticato, ogni volta che il client richiede un report (es. `/daily-report`), deve presentare il "pass" digitale ottenuto.

La Figura 5.1 illustra il percorso fisico e logico che una richiesta compie all'interno dello stack tecnologico Spring Boot.

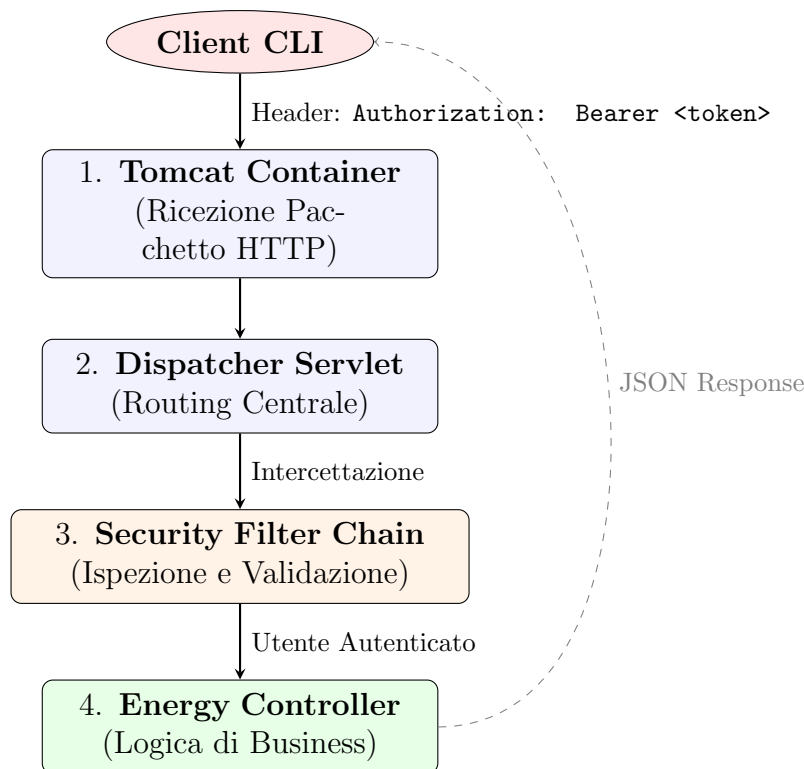


Figura 5.1: Panoramica del flusso di richiesta: dal Client al Controller attraverso i layer di sicurezza.

Come evidenziato nello schema:

- La richiesta viene intercettata dalla **Security Chain** prima ancora che il Controller sappia della sua esistenza.
- Se la catena di sicurezza fallisce, la richiesta viene respinta immediatamente, risparmiando risorse di elaborazione.

5.3 Fase 3: Il Filtro JWT (Deep Dive)

Il componente critico dell'architettura è il **JwtFilter**. Questo filtro agisce come un "doganiere" all'ingresso del sistema. La logica decisionale, illustrata nella Figura 5.3, segue un algoritmo rigoroso per determinare se lasciar passare la richiesta o bloccarla.

Logica decisionale interna al **JwtFilter**. Si noti come l'assenza del token non causi un errore immediato, ma porti al blocco nel filtro successivo (**Authorization Filter**) se la risorsa è protetta.

5.3.1 Dettaglio delle Operazioni

1. **Estrazione:** Il filtro ispeziona l'header HTTP cercando la stringa **Authorization**.
2. **Validazione:** Se trova un token, utilizza la chiave segreta (**Secret Key**) per ricalcolare la firma. Se la firma calcolata corrisponde a quella del token, significa che il dato non è stato alterato.

3. **Contestualizzazione:** Se il token è valido, il filtro crea un oggetto `Authentication` e lo inserisce nel `SecurityContextHolder`.
4. **Autorizzazione:** Solo a questo punto Spring Security controlla se l'utente (ora identificato o rimasto anonimo) ha i permessi per l'URL richiesto. Se un utente anonimo tenta di accedere a `/api/energy`, riceverà un errore **403 Forbidden**.

Capitolo 6

Implementazione Client CLI

Il Client sviluppato (`HeliosClient.java`) funge da console di comando per l'operatore.

6.1 Struttura del Menu

Il client presenta un menu interattivo che guida l'utente attraverso le funzionalità:

```
1 MENU PRINCIPALE:
2     LOGGATO (Token Attivo)
3 0. LOGIN (Richiesto!)
4 1. Report Semplice
5 2. Analisi Batch
6 ...
7 7. Report Economico
8 9. CHIEDI ALL'IA (Ollama)
9 q. Esci
```

Listing 6.1: Esempio di Menu CLI

6.2 Gestione dello Stato

Il client è stato progettato per essere "smart":

- Gestisce automaticamente il ciclo di vita del **Token JWT**.
- Gestisce la persistenza dei **Cookie di Sessione** per i filtri temporali.
- Include logica di formattazione per presentare i JSON ricevuti in modo leggibile.

Capitolo 7

Conclusioni

Il progetto Helios ha dimostrato con successo come un'architettura a microservizi (o client-server disaccoppiata) possa risolvere problemi reali nel dominio dell'energia.

L'integrazione di pattern consolidati come Facade e DTO ha garantito performance e manutenibilità, mentre l'adozione di tecnologie emergenti come le LLM locali ha aggiunto un livello di accessibilità e intelligenza ai dati non comune nei sistemi SCADA tradizionali.

I risultati ottenuti sul dataset di test confermano la validità economica e tecnica della soluzione per contesti industriali.